Capítulo 2. Repositorios y control de cambios en Git

1. Introducción

Los repositorios permiten organizar los archivos de un proyecto y registrar sus modificaciones a lo largo del tiempo. Git ofrece diferentes comandos para crear repositorios, descargar proyectos existentes, guardar cambios y compartirlos mediante plataformas como GitHub.

2. ¿Qué es un repositorio?

Un repositorio es el espacio donde Git mantiene los archivos de un proyecto y el historial de sus cambios. Puede existir en un equipo local o en un servidor remoto.

El repositorio local permite trabajar directamente desde el computador, mientras que el remoto facilita compartir el proyecto y mantener una copia disponible en línea.

3. Crear un repositorio

Para iniciar un repositorio en una carpeta se utiliza:

git init

Este comando inicializa Git en la carpeta actual. Sin embargo, cuando el repositorio ya existe en GitHub, es posible descargarlo junto con su historial mediante el comando git clone.

4. Clonar un repositorio

La clonación crea una copia local de un repositorio remoto. Por ejemplo:

git clone https://github.com/usuario/proyecto.git

Después se entra en la carpeta descargada:

cd proyecto

De esta manera, se puede trabajar localmente y posteriormente enviar los cambios al repositorio remoto.

5. Consultar el estado de los archivos

El comando:

git status

Muestra el estado del repositorio e informa si existen archivos nuevos, modificaciones pendientes de registrar o cambios listos para un commit.

Este comando es útil antes y después de realizar operaciones para comprobar que el trabajo se encuentra en el estado esperado.

6. Preparar los cambios con git add

Cuando se crean o modifican archivos, Git permite seleccionar cuáles se incluirán en el próximo commit.

Por ejemplo:

git add README.md

Para preparar todos los cambios detectados en la carpeta actual:

git add .

Es recomendable revisar previamente el estado del repositorio para evitar agregar archivos temporales o que no formen parte del proyecto.

7. Registrar los cambios con git commit

Después de preparar los archivos, se utiliza:

git commit -m "Agrega el primer capítulo"

El commit registra los cambios preparados y los acompaña de un mensaje descriptivo. Los mensajes claros permiten comprender el historial del proyecto y reconocer qué se hizo en cada etapa.

8. Subir los cambios con git push

Para enviar los commits al repositorio remoto se utiliza:

git push

Este comando permite actualizar el repositorio de GitHub con los cambios registrados localmente. Para que funcione, debe existir una conexión configurada con el repositorio remoto y el usuario debe tener los permisos necesarios.

9. Ejemplo práctico

Un flujo básico para agregar un documento y compartirlo en GitHub puede ser:

1. Crear o modificar el archivo.
2. Ejecutar git status para revisar los cambios.

3. Ejecutar git add nombre-del-archivo.pdf.
4. Ejecutar git commit -m "Agrega documento del capítulo".
5. Ejecutar git push.
6. Consultar el repositorio en GitHub para comprobar que el archivo esté disponible.

Este procedimiento permite registrar el trabajo por etapas y mantener actualizado el proyecto remoto.

10. Conclusión

El uso de repositorios y comandos de control de cambios permite trabajar de forma organizada y conservar un historial del proyecto. Los comandos git clone, git status, git add, git commit y git push son fundamentales para trabajar con Git y compartir los resultados en GitHub.

Referencia

Chacon, S. y Straub, B. (2014). *Pro Git*. Apress. https://git-scm.com/book/es/v2